```
box_office.find_showtime(movie: 'Jaws')
box_office.find_showtime(movie: 'Jaws', zip_code: 97204)
box_office.find_showtime(movie: 'Jaws', city: 'Portland', state: 'OR')
```

When you're testing code that calls such a method, you can use RSpec's hash_including to specify which keys must be present. All three of these find_showtime calls would match the following constraint:

14-customizing-test-doubles/02/setting_constraints.rb

```
expect(box_office).to receive(:find_showtime)
  .with(hash_including(movie: 'Jaws'))
```

**Specify Exactly the Level of Constraint You Need**



Flexible constraints like hash_including make your specs less brittle. Rather than having to give *all* the keys of your hash, you can give just the ones you care about. If the value of an unimportant key changes, your specs needn't fail.

Ruby 2.0 added *keyword arguments* to the language, which provides specific syntax for this style of API:

14-customizing-test-doubles/02/setting_constraints.rb

```
class BoxOffice
  def find_showtime(movie:, zip_code: nil, city: nil, state: nil)
    # ...
  end
end
```

The good news is that the hash_including constraint works just as well with keyword arguments as it does with old-style option hashes.

RSpec also provides a hash_excluding constraint to specify that a hash must *not* include a particular key.

### Custom Logic

When you've written a bunch of constraints, you'll inevitably find yourself repeating the same complex constraint in several specs. Occasionally, you'll need logic that's too involved to express as a simple constraint. In both of these situations, you can supply your own custom logic.

For example, if you have several specs that should specifically call find_showtime with cities in Oregon, you can wrap this constraint up in a custom RSpec matcher like the one you wrote in *A Minimal Custom Matcher*, on page 221:

14-customizing-test-doubles/02/setting_constraints.rb

```
RSpec::Matchers.define :a_city_in_oregon do
  match { |options| options[:state] == 'OR' && options[:city] }
end
```